

Hibernate – Day 04

Employee Entity, Date/Time & Image Handling

1. @GeneratedValue(strategy = GenerationType.AUTO)

We already know that @Id is used to make a field as a Primary Key.

Example:

```
@Id
@GeneratedValue(strategy = GenerationType.AUTO)
private int id;
```

@GeneratedValue is used to automatically generate the primary key value.

We don't need to manually assign the id while inserting the Employee object.

Example:

```
Employee employee = new Employee();

employee.setName("Virat");
```

Here we are not setting:

```
employee.setId(101);
```

Hibernate will automatically generate the ID.

GenerationType.AUTO:

```
@GeneratedValue(strategy = GenerationType.AUTO)
```

Here Hibernate decides which ID generation strategy should be used based on the database and configuration.

2. @Column

@Column is used to customize the mapping between the Java field and the database column.

Example:

```
@Column(name="emp_name")
private String name;
```

Here:

```
Java field name    -> name
```

```
Database column    -> emp_name
```

So Hibernate maps:

```
name -----> emp_name
```

Another example:

```
@Column(length = 20, name="emp_address")
private String fullAddress;
```

Here we are specifying two things:

1. name = "emp_address"

The database column name will be:

```
emp_address
```

instead of:

```
fullAddress
```

2. length = 20

It specifies the length of the String column.

Conceptually:

```
fullAddress -----> emp_address VARCHAR(20)
```

3. @Transient

@Transient tells Hibernate:

"Do not store this field in the database."

Example:

```
@Transient
private String companyHrMail;
```

We can still set the value:

```
employee.setCompanyHrMail("def@Hr");
```

The value will exist inside the Java object.

But Hibernate will NOT create a database column for it.

Example:

```
private String companyName;
```

```
@Transient
private String companyHrMail;
```

Database:

```
companyName      -> Stored
companyHrMail    -> NOT stored
```

Why do we use @Transient?

Sometimes we have data which is required only inside the Java application and we don't want to store that data in the database.

4. DATE AND TIME HANDLING

There are different ways to represent Date and Time in Java.

Earlier, we commonly used:

```
java.util.Date
```

When using java.util.Date, @Temporal can be used to tell Hibernate what part of the date/time should be stored.

@Temporal(TemporalType.DATE)

Example:

```
@Temporal(TemporalType.DATE)
private Date joiningDate;
```

It stores only the DATE.

Example:

```
2026-08-13
```

It does not store the time.

@Temporal(TemporalType.TIME)

Example:

```
@Temporal(TemporalType.TIME)
private Date projectSubmissionTime;
```

It stores only the TIME.

Example:

```
05:00:00
```

It does not store the date.

@Temporal(TemporalType.TIMESTAMP)

Example:

```
@Temporal(TemporalType.TIMESTAMP)
private Date projectAssignmentZone;
```

It stores both DATE and TIME.

Example:

```
2026-08-13 09:00:00
```

IMPORTANT: JAVA 8 DATE/TIME API

In our current Employee class, we are using Java 8 Date/Time classes:

```
LocalDate
LocalTime
LocalDateTime
```

LocalDate:

Used when we need only the DATE.

```
private LocalDate joiningDate;
```

LocalTime:

Used when we need only the TIME.

```
private LocalTime projectSubmissionTime;
```

LocalDateTime:

Used when we need both DATE and TIME.

```
private LocalDateTime projectAssignmentZone;
```

Remember:

```
LocalDate      -> Date only
LocalTime      -> Time only
LocalDateTime  -> Date + Time
```

For LocalDate, LocalTime and LocalDateTime, we do not need to use @Temporal.

@Temporal is mainly used with legacy java.util.Date / java.sql.Date style date types.

5. @Lob – STORING IMAGE

Example:

```
@Lob
private byte[] employeeImage;
```

An image is binary data.

We cannot directly store an image file in a normal String field.

We convert the image into byte[] and then Hibernate stores that binary data in the database.

Flow:



@Lob means Large Object.

Since our Java field is:

```
byte[]
```

Hibernate maps it to a binary large object, generally a BLOB.

COMPLETE EMPLOYEE CLASS

```
package com.nit.Entity;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Lob;
import jakarta.persistence.Table;
import jakarta.persistence.Transient;

import lombok.Data;
```

```

@Entity
@Table(name="employee_details")
@Data
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private int id;

    @Column(name="emp_name")
    private String name;

    @Column(length = 20, name="emp_address")
    private String fullAddress;

    private String companyName;

    @Transient
    private String companyHrMail;

    @Column(name="emp_joinDate")
    private LocalDate joiningDate;

    @Column(name="submit")
    private LocalTime projectSubmissionTime;

    @Column(name="assignOn")
    private LocalDateTime projectAssignmentZone;

    @Lob
    private byte[] employeeImage;
}

```

HIBERNATE-CNF.XML

We are NOT going to make any changes in hibernate-cnf.xml just because we modified the Employee class.

We are using the same Hibernate configuration as before.

The mapping class will also remain the same:

```
<mapping class="com.nit.Entity.Employee"/>
```

Hibernate will automatically read the new fields and annotations from the Employee entity.

If we are using:

```
<property name="hibernate.hbm2ddl.auto"
value="create-drop"/>
```

Hibernate can create the table based on the updated Employee entity mapping.

INSERTING LOCALDATE, LOCALTIME AND LOCALDATETIME

We already know:

```
Configuration
SessionFactory
buildSessionFactory()
Session
openSession()
Transaction
persist()
get()
```

Now we will see how to insert the new values.

1. LocalDate

Employee contains:

```
private LocalDate joiningDate;
```

We can create LocalDate using:

```
LocalDate.of(2020, 11, 12)
```

Then:

```
employee.setJoiningDate(
    LocalDate.of(2020, 11, 12)
);
```

This represents:

```
12-November-2020
```

Only the DATE is represented.

2. LocalTime

Employee contains:

```
private LocalTime projectSubmissionTime;
```

We can create LocalTime using:

```
LocalTime.of(5, 0)
```

Then:

```
employee.setProjectSubmissionTime(  
    LocalDateTime.of(5, 0)  
);
```

This represents:

```
05:00
```

Only the TIME is represented.

3. LocalDateTime

Employee contains:

```
private LocalDateTime projectAssignmentZone;
```

We can create LocalDateTime using:

```
LocalDateTime.of(2026, 8, 13, 9, 0)
```

Then:

```
employee.setProjectAssignmentZone(  
    LocalDateTime.of(2026, 8, 13, 9, 0)  
);
```

This represents:

```
13-August-2026 09:00
```

It contains:

```
DATE + TIME
```

READING IMAGE FROM COMPUTER

Suppose our image is located at:

```
C:\Users\pjpra\Downloads\pic\rengokuPic.jpg
```

First create a Path:

```
Path path = Paths.get(  
    "C:\\Users\\pjpra\\Downloads\\pic\\rengokuPic.jpg"  
);
```

Then:

```
byte[] imageBytes = Files.readAllBytes(path);
```

Files.readAllBytes() reads the complete image from the computer and converts it into byte[].

Flow:

```
    rengokuPic.jpg
      |
      v
Files.readAllBytes()
      |
      v
    byte[]
```

Now we have the image in Java memory.

SETTING IMAGE INTO EMPLOYEE OBJECT

We have:

```
byte[] imageBytes
```

We assign it to the Employee object:

```
employee.setEmployeeImage(imageBytes);
```

Now the Employee object contains the image as binary data.

INSERTING IMAGE INTO DATABASE

When we call:

```
session.persist(employee);
```

Hibernate stores the Employee object.

The image is stored through:

```
@Lob
private byte[] employeeImage;
```

Flow:

```
Image File
  |
  v
readAllBytes()
  |
  v
  byte[]
  |
  v
setEmployeeImage()
```

```

      |
      v
session.persist()
      |
      v
    Hibernate
      |
      v
      BLOB
      |
      v
    Database

```

READING EMPLOYEE FROM DATABASE

To retrieve an Employee:

```

Employee employee2 =
    session.get(Employee.class, 1);

```

Here:

```

Employee.class

```

tells Hibernate which entity we want.

And:

```

1

```

is the primary key.

Hibernate retrieves the Employee whose ID is 1.

READING IMAGE FROM DATABASE

After getting the Employee:

```

Employee employee2 =
    session.get(Employee.class, 1);

```

We can get the image using:

```

byte[] employeeImage =
    employee2.getEmployeeImage();

```

Now the flow is:

```

Database

```

```

|
v
BLOB
|
v
Hibernate
|
v
Employee object
|
v
getEmployeeImage()
|
v
byte[]

```

WRITING IMAGE BACK TO COMPUTER

Create the destination path:

```

Path path1 = Paths.get(
    "C:\\Users\\pjpra\\Downloads\\pic\\rengokuReturnPic.jpg"
);

```

Then:

```

Files.write(path1, employeeImage);

```

Files.write() takes the byte[] and creates the image file.

Flow:

```

byte[]
|
v
Files.write()
|
v
rengokuReturnPic.jpg

```

COMPLETE App.java

```

package com.nit.Day03;

import java.io.IOException;
import java.nio.file.Files;

```

```

import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

import com.nit.Entity.Employee;

public class App {

    @SuppressWarnings("removal")
    public static void main(String[] args) throws IOException {

        Configuration cnfig = new Configuration();

        cnfig.configure("resources/hibernate-cnf.xml");

        SessionFactory factory =
            cnfig.buildSessionFactory();

        System.out.println(factory);

        Employee employee = new Employee();

        employee.setName("Virat");
        employee.setFullAddress("Banglore");
        employee.setCompanyName("def");
        employee.setCompanyHrMail("def@Hr");

        // LocalDate

        employee.setJoiningDate(
            LocalDate.of(2020, 11, 12)
        );

        // LocalTime

        employee.setProjectSubmissionTime(
            LocalTime.of(5, 0)
        );
    }
}

```

```

);

// LocalDateTime

employee.setProjectAssignmentZone(
    LocalDateTime.of(2026, 8, 13, 9, 0)
);

// Reading image from computer

Path path = Paths.get(
    "C:\\Users\\pjpra\\Downloads\\pic\\rengokuPic.jpg"
);

byte[] imageBytes =
    Files.readAllBytes(path);

// Setting image into Employee object

employee.setEmployeeImage(imageBytes);

Session session = factory.openSession();

Transaction transaction =
    session.beginTransaction();

// Insert Employee

session.persist(employee);

System.out.println(
    "Get the image back from the database"
);

// Read Employee from database

Employee employee2 =
    session.get(Employee.class, 1);

```

```

// Get image from Employee object

byte[] employeeImage =
    employee2.getEmployeeImage();

// Destination path

Path path1 = Paths.get(
    "C:\\Users\\pjpra\\Downloads\\pic\\rengokuReturnPic.jpg"
);

// Write image to computer

Files.write(path1, employeeImage);

System.out.println("Image stored");

transcation.commit();

session.close();
}
}

```

FINAL FLOW

INSERT

```

Image on Computer
    |
    v
Files.readAllBytes()
    |
    v
byte[]
    |
    v
setEmployeeImage()
    |
    v
session.persist()

```

```

      |
      v
    Hibernate
      |
      v
      BLOB
      |
      v
    Database
  
```

READ

```

      Database
      |
      v
      BLOB
      |
      v
    Hibernate
      |
      v
    Employee Object
      |
      v
    getEmployeeImage()
      |
      v
      byte[]
      |
      v
    Files.write()
      |
      v
    Image on Computer
  
```

QUICK REVISION

@GeneratedValue	-> Automatically generates primary key value.
GenerationType.AUTO	-> Hibernate decides the ID generation strategy.
@Column(name="...")	-> Changes the database column name.
@Column(length=20)	-> Specifies the length of a String column.
@Transient	-> Field will not be stored in the database.

@Temporal(TemporalType.DATE)	-> Date only for legacy Date types.
@Temporal(TemporalType.TIME)	-> Time only for legacy Date types.
@Temporal(TemporalType.TIMESTAMP)	-> Date + Time for legacy Date types.
LocalDate	-> Date only.
LocalTime	-> Time only.
LocalDateTime	-> Date + Time.
@Lob + byte[]	-> Used to store binary large data such as an image.
Files.readAllBytes()	-> Reads file and converts it into byte[].
Files.write()	-> Converts byte[] back into a file.

ONE-LINE MEMORY TRICK

```

    LocalDate    -> DATE
    LocalTime    -> TIME
    LocalDateTime -> DATE + TIME
    @Transient   -> DON'T STORE
    @Lob + byte[] -> STORE BINARY DATA / IMAGE

```

Image:

```
File -> byte[] -> Hibernate -> BLOB -> Database
```

Read:

```
Database -> BLOB -> Hibernate -> byte[] -> File
```